

Parametrized Algorithms and Complexity: Lossy Kernelization

Andreas Panayi
A.A.MA.

Dept. of Mathematics
National and Kapodistrian University of Athens

June 15, 2025

Contents

- 1 Introduction
 - Establishing the context
 - Motive
 - Previous Work
- 2 Definitions and Frame
 - Parametrized Optimization Problems
 - Parametrized Approximation
- 3 Foundational Results
- 4 Approximate Kernels Design
 - Problem 1: Connected Vertex Cover

Contents

- 5 More α -approximation Kernels
- 6 Lower Bounds Framework
 - Existing Work
 - Extension To Approximation Kernels
- 7 Discussion
 - Benefits of Approximate Kernels
 - Lower Bounds Results
 - Roadmap for Future Problems
- 8 Bibliography
 - References

Part 1:

INTRODUCTION

Building On Kernelization

- We will present a **new framework** for analyzing the performance of pre-processing algorithms.
- This framework builds on the notion of *kernelization* from parameterized complexity.

Need For Innovation

- Despite the success of kernelization, the basic definition has an important drawback: *it does not combine well with approximation algorithms or with heuristics.*
- Serious problem, since the goal of parameterized algorithms (of generally any algorithmic paradigm) is to **eventually solve the given input instance.**

Non Formal Introduction

- The new definitions combine well with approximation algorithms and heuristics.
- The key new definition is that of a polynomial size α -approximate kernel.
- The starting point of our new framework is an extension of kernelization to *optimization problems*.

Idea Of Approximate Kernels

Loosely speaking, polynomial size α -approximate kernel is a polynomial time (pre-processing) algorithm that:

1. Takes as input an instance (I, k) to a parameterized problem, and outputs another instance (I', k') to the same problem, such that $|I'| + k' \leq k^{O(1)}$ ¹.
2. For every $c \geq 1$, a c -approximate solution s' to the pre-processed instance (I', k') can be turned in polynomial time into a $(c \cdot \alpha)$ -approximate solution s to the original instance (I, k) .

¹Generally it could be $\leq f(k)$, when f is linear, quadratic, polynomial etc we say that the (size of the) kernel is linear, quadratic, polynomial respectively

Motive: Positive Results

Some problems (for example: Connected Vertex Cover, Disjoint Cycle Packing and Disjoint Factors) which are **known not to admit any polynomial size kernels** unless $\text{NP} \subseteq \text{coNP}/\text{Poly}$ are shown (later in the slides) to **have α -approximate kernels of polynomial size**.

Motive: Negative Results

On the negative side it has been proved that Longest Path parameterized by the length of the path and Set Cover parameterized by the universe size do not admit even an α -approximate kernel of polynomial size, for any $\alpha \geq 1$, unless $\text{NP} \subseteq \text{coNP}/\text{Poly}$.

Previous Work

Fellows et al. [Fel+12] proposed a definition of lossy kernelization called α -fidelity kernels.

Essentially, an α -fidelity kernel is a polynomial time pre-processing procedure such that an optimal solution to the reduced instance translates to an α -approximate solution to the original.

Downsides Of Previous Work

For many reasons (beyond the scope of this presentation) we strongly believe that the notion of α -approximate kernels is a better model of lossy kernelization than α -fidelity kernels are.

Note: it seems that the existence of α -approximate kernels can be used to establish the existence of α -fidelity kernels and vice versa.

Part 2:

DEFINITIONS AND FRAME

Parametrized Optimization Problem

A **parameterized optimization** (minimization or maximization) **problem** Π is a computable function

$$\Pi : \Sigma^* \times \mathbb{N} \times \Sigma^* \rightarrow \mathbb{R} \cup \{\pm\infty\}.$$

The instances of a parameterized optimization problem Π are pairs $(I, k) \in \Sigma^* \times \mathbb{N}$, and a solution to (I, k) is simply a string $s \in \Sigma^*$, such that $|s| \leq |I| + k$.

The **value** of the solution s is $\Pi(I, k, s)$.

FPT Algorithms

A parameterized optimization problem Π is **fixed-parameter tractable** (FPT) if there exists an algorithm that, on any input (I, k) ,

- ① outputs an optimal solution s for (I, k) , and
- ② runs in time $f(k) \cdot n^{O(1)}$,
where $n = |I|$ and f is a computable function of k .

Pre-Processing Algorithms (a)

A polynomial time pre-processing algorithm A for a parameterized optimization problem Π is a **pair** of polynomial time algorithms.

The **first** one is called the **reduction algorithm**, and computes a map $R_A : \Sigma^* \times \mathbb{N} \rightarrow \Sigma^* \times \mathbb{N}$.

Given as input an instance (I, k) of Π the reduction algorithm outputs another instance $(I', k') = R_A(I, k)$.

Pre-Processing Algorithms (b)

The **second** algorithm is called the **solution lifting algorithm**.

This algorithm takes as input an instance $(I, k) \in \Sigma^* \times \mathbb{N}$ of Π , the output instance (I', k') of the reduction algorithm, and a solution s' to the instance (I', k') .

The solution lifting algorithm works in time polynomial in $|I|, k, |I'|, k'$ and s' , and outputs a solution s to (I, k) ².

$$\text{size}_A(k) = \sup\{|I'| + k' : (I', k') = R_A(I, k), I \in \Sigma^*\}.$$

²If s' is an optimal solution to (I', k') then s is an optimal solution to (I, k)

Kernelization

A **kernelization** (or **kernel**) for a parameterized optimization problem Π is a **polynomial time pre-processing algorithm** A such that **size _{A}** is **upper bounded** by a computable function $g : \mathbb{N} \rightarrow \mathbb{N}$.

Approximate Kernelization: Revisit

Let $a \geq 1$ an α -approximate kernel has to:

- ① Take an instance I with parameter k as input
- ② Run in polynomial time
- ③ Produce a new instance I' with parameter k' s.t. $|I'| + k' \leq g(k)$

This function $g(k)$ is the size of the kernel.

Definition extends to the next slide...

Approximate Kernalization: Revisit

Also a solution s' to the instance (I', k') should be useful for finding a good solution s to the instance (I, k) .

More precisely when k is ...

Structural Parameter

If we are working with a structural parameter k (both for min and max problems) we require the following from α -approximate kernels:
For every $c \geq 1$, a c -approximate solution s' to I' can be transformed in polynomial time into a $(c \cdot \alpha)$ -approximate solution to I .

Minimization Problems

For every $c \geq 1$, a c -approximate solution s' to I' can be transformed in polynomial time into a $(c \cdot \alpha)$ -approximate solution s to I . However, if the quality of s' is “worse than” k' , or $(c \cdot \alpha) \cdot \text{OPT}(I) > k$, the algorithm that transforms s' into s is allowed to fail.

Maximization Problems

For every $c \geq 1$, if s' is a c -approximate solution s' to I' or the quality of s' is at least k'/c , then s' can be transformed in polynomial time into a $(c \cdot \alpha)$ -approximate solution s to I .

However, if $OPT(I) > k$ then instead of being a $(c \cdot \alpha)$ -approximate solution s to I , the output solution s can be any solution of quality at least $k/(c \cdot \alpha)$.

Strict α -approximate pre-processing Algorithm

An α -approximate polynomial time pre-processing algorithm is said to be **strict** if, for every instance (I, k) , reduced instance $(I', k') = R_A(I, k)$ and solution s' to (I', k') , the solution s to (I, k) output by the solution lifting algorithm when given s' as input satisfies the following:

Strict α -approximate pre-processing Algorithm

- If Π is a minimization problem then

$$\frac{\Pi(I, k, s)}{\text{OPT}(I, k)} \leq \max \left\{ \frac{\Pi(I', k', s')}{\text{OPT}(I', k')}, \alpha \right\}.$$

- If Π is a maximization problem then

$$\frac{\Pi(I, k, s)}{\text{OPT}(I, k)} \geq \min \left\{ \frac{\Pi(I', k', s')}{\text{OPT}(I', k')}, \frac{1}{\alpha} \right\}.$$

Strict α -approximate pre-processing algorithms can be chained.

Safe α -reduction Rule

A reduction rule is said to be α -safe for Π if there exists a solution lifting algorithm, such that the rule together with the solution lifting algorithm constitute a strict α -approximate polynomial time pre-processing algorithm for Π .

PSAKS

Sometimes we obtain a $(1 + \varepsilon)$ -approximate kernel of polynomial size for every $\varepsilon > 0$ (here the exponent and the constants of the polynomial may depend on ε).

We call such a kernel a Polynomial Size Approximate Kernelization Scheme, and abbreviate it as PSAKS.

EPSAKS

If only the constants of the polynomial $g(k)$ and not the exponent depend on ε , we say that the PSAKS is efficient. In short EPSAKS.

Part 3:

FOUNDATIONAL RESULTS

Theorem 1: Exact Solutions

We can prove that a decidable parameterized optimization problem Π is FPT if and only if it admits a kernel.

Proof Of Theorem 1

Run (I, k) on the reduction algorithm to obtain a new instance (I', k') of size bounded by a function $g(k)$.

Run brute force to obtain optimal solution s' to (I', k') in time upper bounded by a function $g'(k)$.

Finally one can use the solution lifting algorithm to obtain an optimal solution s to (I, k) .

Proof Of Theorem 1 $\Rightarrow$

Reduction Algorithm:

On input (I, k) the reduction algorithm runs the FPT algorithm for n^{c+1} steps.

If the FPT algorithm terminates after at most n^{c+1} steps, the reduction algorithm outputs an instance (I', k') of constant size thus upper bounded by a constant.

If the FPT algorithm does not terminate after n^{c+1} steps the reduction algorithm halts and outputs the instance (I, k) . In this case $f(k)n^c > n^{c+1}$, which implies that $f(k) > |I|$.

Proof Of Theorem 1 $\Rightarrow$

Lifting Algorithm:

If the reduction algorithm output (I, k) then the solution lifting algorithm returns the same solution that it gets as input.

If the reduction algorithm output (I', k') this means that the FPT algorithm terminated in polynomial time, which means that the solution lifting algorithm can use the FPT algorithm to output an optimal solution to (I, k) in polynomial time (regardless of the solution to (I', k') it gets as input).

Theorem 2: General Case

For every $\alpha \geq 1$ and decidable parameterized optimization problem Π , Π admits a fixed parameter tractable α -approximation algorithm if and only if Π has an α -approximate kernel.

Theorem 3: Polynomial Time

For every $\alpha \geq 1$ and decidable parameterized optimization problem Π , Π admits a polynomial time α -approximation algorithm if and only if Π has an α -approximate kernel of constant size.

Part 4:

APPROXIMATE KERNELS DESIGN

Problem Definition

The parameterized optimization problem Connected Vertex Cover (CVC) is defined as follows.

$$\text{CVC}(G, k, S) = \begin{cases} \infty & \text{if } S \subseteq V(G) \text{ is not a connected vertex} \\ & \text{cover of the graph } G \\ \min\{|S|, k + 1\} & \text{otherwise} \end{cases}$$

We show that CVC has a polynomial size strict α -approximate kernel for every $\alpha > 1$ (PSAKS).

CVC Approximation Kernel

Let (G, k) be the input instance.

Wlog assume that the input graph G is connected.

Let $d := \lceil \frac{\alpha}{\alpha-1} \rceil$.

For a graph G and an integer k , define

$$H := \{v \in V(G) \mid \deg_G(v) \geq k + 1\},$$

$$I := \{v \in V(G) \setminus H \mid N_G(v) \subseteq H\},$$

$$R := V(G) \setminus (H \cup I).$$

CVC: Reduction Rule 1

Reduction Rule 1:

Let $v \in I$ be a vertex of degree $D \geq d$.

- ① *Delete $N_G[v]$ from G*
- ② *Add a vertex w such that the neighborhood of w is $N_G(N_G(v)) \setminus \{v\}$.*
- ③ *Then add k degree 1 vertices $v_1, \dots, v_k$ whose neighbor is w .*
- ④ *Output this graph G' , together with the new parameter $k' = k - (D - 1)$.*

CVC: Reduction Rule 2

A *false twin* of a $v \in V(G)$ is a $u \in V(G)$ s.t. $uv \notin E(G)$ and $N(u) = N(v)$.

Reduction Rule 2.

If a vertex v has at least $k + 1$ false twins, then remove v .

Output $G' = G - v$ and $k' = k$.

CVC: Correctness Of Reduction rules

Lemma 0:

- Reduction Rule 1 is α -safe
 - If S' is a connected vertex cover of G' of size at most k' , the algorithm returns the set $S = (S' \setminus \{w, v_1, \dots, v_k\}) \cup N_G[v]$.
- Reduction Rule 2 is 1-safe

CVC: Rules Lead To Kernel

Lemma 1:

Let (G, k) be an instance irreducible by rules 1 and 2, such that $\text{OPT}(G, k) \leq k$. Then $|V(G)| \leq O(k^d + k^2)$.

Proof Sketch:

We count the vertices in H, I, R .

Approximation Kernel For CVC

Theorem:

Connected Vertex Cover admits a strict time efficient PSAKS with $O(k^{\lceil \frac{\alpha}{\alpha-1} \rceil} + k^2)$ vertices.

Proof Sketch

Proof Sketch:

- Apply rules 1 and 2 exhaustively
- If the reduced graph G has more than $O(k^d + k^2)$ by Lemma 1 $\text{OPT}(G, k) = k + 1$, so return any connected vertex cover of G as an optimal solution
- Otherwise the reduced graph has at most $O(k^d + k^2)$ vertices, and the size bound follows.

The entire reduction procedure runs in polynomial time (independent of α), hence the PSAKS is time efficient.

Part 5:

MORE α -APPROXIMATION KERNELS

Exploiting Existing Results

Some of the existing approximation algorithms and FPT approximation algorithms can be re-interpreted as:

1. First **computing an α -approximate kernel**
2. Then **running a brute force search or an approximation algorithm** on the reduced instance

Examples

- Partial Vertex Cover *admits a strict time and size efficient* PSAKS
- Steiner Tree *parameterized by the number of terminals admits a* PSAKS
- Optimal Linear Arrangement *parameterized by vertex cover has a* $(1 + \varepsilon)$ -approximate kernel of size $O(\frac{1}{\varepsilon} 2^k k^4)$

Part 6:

LOWER BOUNDS FRAMEWORK

Previous Work

- Bodlaender et al. [BJK11] defined cross composition and showed that if an NP-hard language L admits a cross composition into a parameterized (decision) problem Π and Π admits a polynomial kernel, then L has an **OR-distillation algorithm**.
- Fortnow and Santhanam [FS11] proved that if an NP-hard language L has an OR-distillation, then $\text{NP} \subseteq \text{coNP}/\text{Poly}$.

So o a cross composition from L to Π proves that Π **does not have a polynomial kernel** unless $\text{NP} \subseteq \text{coNP}/\text{Poly}$.

Extension To Approximation Kernels

- In order to prove lower bounds on the size of α -approximate kernels, we generalize the notion of cross compositions to α -gap cross compositions.
- Then by introducing the notions of α -approximate polynomial parameter transformation and α -approximate compression, combining them with the more general result by Dell and Melkebeek [DM10] we can show a very similar result, yielding essentially the same conclusion.

Lower Bounds Theorem

Theorem:

Let L be an NP-hard language and Π be a nice³ parameterized optimization problem. If L α -gap cross composes to Π , and Π has a polynomial sized α -approximate compression, then $\text{NP} \subseteq \text{coNP}/\text{Poly}$.

³ Π is called nice when it is polynomial time verifiable, in other words, the function Π is computable in polynomial time.

Part 7:

DISCUSSION

Why Approximate Kernels Matter

- **Stronger Approximation:**

By allowing a small constant factor $\alpha > 1$, we often beat the best known polynomial time approximation guarantees.

- **Smaller Kernel Size:**

Exact kernels can be large (e.g. polynomial of high degree or even superpolynomial). Approximate kernels shrink the output to a polynomial (often linear) function of the parameter k .

Summary of Lower Bound Results

- **Longest Path** (parameter: path length):
 - No polynomial-size α -approximate kernel for any constant $\alpha \geq 1$, unless $\text{NP} \subseteq \text{coNP}/\text{poly}$.
- **Set Cover** (parameter: universe size n):
 - No polynomial-size α -approximate kernel for any constant $\alpha \geq 1$, unless $\text{NP} \subseteq \text{coNP}/\text{poly}$.
- **Hitting Set** (parameter: universe size n):
 - No $O(2^{(\log n)^c})$ -approximate kernel of size $n^{O(1)}$ for any $c < 1$, unless ETH fails.

Kernelization and Approximation Roadmap

- For problems that provably do not admit polynomial-size kernels but do admit constant-factor approximation algorithms:
 - One should search for **PSAKSes**.
- For problems with PSAKSes:
 - One should search for **efficient PSAKSes**.
- For problems with no polynomial kernel **and** no constant-factor approximation:
 - One may look for a **constant-factor approximate kernel** of polynomial size.
- For problems that do have polynomial kernels:
 - One can search for **approximate kernels** that are even smaller.

Part 7:

BIBLIOGRAPHY

References I

- [BJK11] Hans L. Bodlaender, Bart M. P. Jansen, and Stefan Kratsch. “Cross-Composition: A New Technique for Kernelization Lower Bounds”. In: *28th International Symposium on Theoretical Aspects of Computer Science (STACS)*. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 2011, pp. 165–176. DOI: [10.4230/LIPIcs.STACS.2011.165](https://doi.org/10.4230/LIPIcs.STACS.2011.165).
- [DM10] Holger Dell and Dieter van Melkebeek. “Satisfiability allows no nontrivial sparsification unless the polynomial-time hierarchy collapses”. In: *Proceedings of the 42nd ACM Symposium on Theory of Computing (STOC)*. 2010, pp. 251–260. DOI: [10.1145/1806689.1806726](https://doi.org/10.1145/1806689.1806726).

References II

- [Fel+12] Michael R. Fellows et al. “Parameterized approximation via fidelity preserving transformations”. In: *Automata, Languages, and Programming (ICALP 2012), Part I*. Vol. 7391. Lecture Notes in Computer Science. Springer, 2012, pp. 351–362. DOI: [10.1007/978-3-642-31594-7_30](https://doi.org/10.1007/978-3-642-31594-7_30).
- [FS11] Lance Fortnow and Rahul Santhanam. “Infeasibility of instance compression and succinct PCPs for NP”. In: *Journal of Computer and System Sciences* 77.1 (2011), pp. 91–106. DOI: [10.1016/j.jcss.2010.05.004](https://doi.org/10.1016/j.jcss.2010.05.004).
- [Lok+16] Daniel Lokshtanov et al. “Lossy Kernelization”. In: *arXiv preprint arXiv:1604.04111* (2016).

References III

- [Lok+17] Daniel Lokshtanov et al. “Lossy Kernelization”. In: *Proceedings of the 49th Annual ACM SIGACT Symposium on Theory of Computing (STOC)*. 2017, pp. 224–237. DOI: [10.1145/3055399.3055456](https://doi.org/10.1145/3055399.3055456).

Thanks

Thanks for your attention 😊

Special *thanks* to Dr. Evangelos Protopapas